How Knowledge Updating Ripples into "Vulnerable" Knowledge?

Anonymous ACL submission

Abstract

Updating a language model's knowledge via fine-tuning or editing is necessary for keeping models up-to-date, yet it often triggers undesired ripple effects such as catastrophic forgetting and newly induced hallucinations. *Heng* **[also talk about the bad impact of old knowledge, including hallucination etc.]** Despite growing attention to side effects, we still lack a systematic understanding of which knowledge is most likely to be corrupted and how far the disturbance propagates. We address this gap by constructing a large knowledge graph from a real-world corpus: a verified functional knowledge graph with paired factual triplets, stratified by node in-degree as a proxy for knowledge "popularity". We then perform controlled single-fact knowledge updates using parameter-efficient tuning and quantify ripple effects over increasing graph distance using an error propagation rate that captures hallucinations. Our results show that ripple effects routinely extend well beyond the edited fact, overturning unrelated knowledge even at distance exceeding five hops. Strikingly, high-connectivity hubs, which are widely connected, "popular" knowledge is significantly more likely to flip from correct to incorrect than long-tail knowledge, contradicting the common intuition that popular knowledge is inherently robust. Meanwhile, updates can also distort model calibration: many facts that were wrong before remain wrong after updating but become more confident, producing high-confidence hallucinations. Finally, leveraging graph structure yields an effective mitigation: anchoring updates with a small set of high-connectivity hub facts *Heng* **[these are very difficult to understand without definition]** (selected via our popularity proxy) substantially dampens long-range propagation and reduces hallucinations, outperforming random anchoring, especially at larger distances. *Kathy* **[Abstract is quite long. I would reduce and save some information to first bring up in introduction. Abstract doesn't have to have everythign.]**

1 Introduction

Large language models (LLMs) are powerful in storing and utilizing knowledge, yet their knowledge needs to continuously evolve with the changing world. This has led to widespread use of post-training techniques such as fine-tuning and knowledge editing *Heng* **[add citations]** to update model knowledge. However, updating a single piece of knowledge often introduces unintended side effects, where changes propagate beyond the target fact and distort other, seemingly unrelated knowledge. *Kathy* **[This is the continual learning problem and appropriate references should be inserted.]**This phenomenon, commonly referred to as the *ripple effect*, highlights a fundamental challenge: knowledge updating in LLMs is not a local operation, but can trigger global changes in an interconnected system.

While ripple effects have been observed, it remains unclear *what governs their propagation across knowledge*. A natural hypothesis is that such effects are driven by semantic or lexical similarity between facts. Prior work and empirical observations suggest that models are prone to confusion among semantically related entities, leading to the expectation that errors may propagate along similarity (cite). At the same time, another plausible hypothesis is that different types of knowledge exhibit different levels of robustness. In particular, long-tail knowledge, which is less frequently observed during training, is often assumed to be more fragile and thus more susceptible to corruption (cite). *Heng* **[these descriptions are all too abstract]** In this work, we aim to test these hypotheses and answer a central question: *Kathy* **[The above section is nicely written so long as there are ties to continual learning. The difference may be in looking at relatedness in language. I don't think semantic relatedness is generally considered, though we should check.]**

What determines the ripple effect in knowledge updating?

To systematically study how relationships between knowledge affect ripple propagation, we construct a structured knowledge graph grounded in real-world corpora. This allows us to explicitly model connections between facts and quantify how updates propagate along these connections. Within this framework, each piece of knowledge corresponds to a node, and relationships between facts define the edges, enabling us to analyze how ripple effects unfold along the graph structure.

Using this framework, we analyze how ripple effects vary across different parts of the graph. We observe that nodes with higher connectivity are significantly more vulnerable to corruption than sparsely connected ones, where connectivity reflects popularity of knowledge. Yuji[define popularity explicitly and accurately] Moreover, highly connected nodes not only exhibit greater susceptibility, but also play a dominant role in propagating errors to other parts of the knowledge space.

Interestingly, the connectivity of a node in the graph aligns with how frequently a piece of knowledge is referenced or shared across facts, allowing us to interpret it as a proxy for knowledge popularity. Kathy[OK I see it here. But use italics and more formally define. I think you do want to distinguish from frequency. I thought in Bengal talk you said it was not frequency.] Under this interpretation, highly connected nodes correspond to widely shared or commonly used knowledge, while sparsely connected nodes correspond to long-tail knowledge. From this perspective, the observed behavior is counterintuitive: widely shared (i.e., popular) knowledge is generally expected to be more robust, Heng[not sure what you mean by knowledge is robust] yet we find it to be more vulnerable and more influential in error propagation. This finding contradicts both the similarity-based explanation Kathy[what is this? explain and cite.]and the common intuition that long-tail knowledge is the primary source of fragility, Kathy[cite this about long-tailed also.]suggesting instead that ripple effects are governed by how centrally knowledge is connected. This observation naturally leads to a deeper question:

Why is hub knowledge both more vulnerable and more influential?

To answer this, we analyze the internal representations of LLMs from two complementary perspectives. First, we show that popular knowledge exhibits significantly narrower decision margins, indicating fragile decision boundaries that are easily perturbed by updates. Kathy[This previous sentence not clear. What are narrower decision margins?]Second, we demonstrate that hub nodes participate in stronger attention connectivity, forming effective pathways for propagating updates across distant knowledge. Together, these results suggest that the same structural property, connectivity, simultaneously determines both the stability of knowledge representations and the pathways through which errors spread. Finally, this understanding raises a practical question:

Can we leverage hub knowledge to control ripple effects?

If highly connected nodes amplify error propagation, they may also serve as anchors to stabilize the model. Building on this insight, we propose a connectivity-aware regularization strategy that selectively anchors hub knowledge during updates. We show that this approach effectively reduces long-range ripple effects and improves overall stability, outperforming connectivity-agnostic baselines. This demonstrates that the same mechanism that causes instability can also be harnessed for control.

Overall, our work reveals a unifying principle:

Node popularity, as a proxy for structural connectivity, governs both the vulnerability and propagation of ripple effects in LLMs.

By identifying connectivity as the key factor underlying knowledge distortion, we provide both a deeper understanding of knowledge dynamics in LLMs and a practical pathway toward more reliable knowledge updating.

Kathy[Overall this intro is very interesting. Just needs tightening up of terminology use, more citations, and definitions.]

2 Related Work

Yuji[In related work, we need to clarify several points: 1. why we use parameter-efficient fine-tuning (minimal disturbance of model utility and exploration into ripple effects under small updates (or clarify it in the setting section?) 2. the difference between expected ripple effect and unexpected ripple effect.)]

2.1 Knowledge Injection in LLMs

Knowledge updating has become a key approach to enhance LLMs' factuality, with methods spanning fine-tuning (Zhu et al., 2020), prompt engineering

(Wei et al., 2022), and knowledge graph (KG) integration (Pan et al., 2023; Yasunaga et al., 2021).

Bosselut et al. (2019) fine-tuned LLMs with KG triples to improve question-answering accuracy, while Yasunaga et al. (2021) integrated KG embeddings into LLM layers to reduce hallucinations. Yuji[Revise this paragraph to emphasize that previous work focuses more on expected ripple effects and explores more about how similarity or logic affects ripple effects] However, most studies only evaluate post-injection performance via task accuracy (e.g., answer correctness) and rarely investigate how Weibing[controlled Weibing[artificial] updates] Yuji[not necessarily false, common knowledge updating also causes it.] distort model behavior—let alone their ripple effects on interconnected knowledge.

Weibing[

2.2 Continual Learning and Catastrophic Forgetting

Our investigation of ripple effects is fundamentally connected to the rich literature on Continual Learning (CL) and catastrophic forgetting (??). Traditional CL research focuses on sequential learning of new tasks without degrading performance on previously learned ones, proposing regularizers like Elastic Weight Consolidation (EWC) (**Kirkpatrick et al., 2017**), episodic memory replay (e.g., GEM, A-GEM) (??), and architecture-expansion methods. While CL typically studies macroscopic performance degradation across entire datasets or task distributions, our work adapts this framework to the microscopic level: we use Weibing[artificial] injections as controlled, single-fact perturbations to trace exactly *how* catastrophic forgetting propagates along topological boundaries within the model's internal knowledge graph.]

3 The RippleEval Benchmark

Yuji[Just updated a new version of the benchmark, will fix more details in more rounds later]

To comprehensively analyze how factual updates affect related knowledge after post-training, we construct **RippleEval**, a QA-based benchmark grounded in verified factual relations.

Each QA sample in RippleEval corresponds to an underlying factual relation (s, r, o), where s and o are entities and r is a relation. We use these grounded relations to connect QA samples because QA expressions themselves are not stable units for defining factual dependency: the same fact can be asked in different ways, while unrelated facts may share similar question templates. Thus, the graph is used to define factual connections among QA samples, while model training and evaluation are still performed through natural-language QA.

Grounded Graph Construction. We construct an entity-level factual graph and use it to organize QA samples. In this graph, each node is an entity, and each directed edge represents a verified factual relation (s, r, o) from subject entity s to object entity o under relation r. Each verified edge is then verbalized into a QA sample for training and evaluation.

Relation Constraints. To make factual connections and downstream effects well-defined, we restrict RippleEval to relations with a unique or primary expected answer. Existing benchmarks such as MQUAKE (Zhong et al., 2023) and RippleEdits (Cohen et al., 2023) evaluate whether edited facts lead to correct multi-hop or ripple-effect consequences. However, their goal is broad evaluation of editing consistency rather than controlling whether each subject-relation pair has a single expected object answer. In RippleEval, we impose this constraint to make the affected factual neighbors and their expected answers unambiguous.

Specifically, for each subject-relation pair (s, r), we require one expected object answer o. Relations such as *CapitalOf* and *DevelopedByPrimary* satisfy this requirement, while relations such as *HasChild* are excluded because they can have many valid objects. Note that it only requires each subject-relation pair to have a single expected answer. This constraint allows us to define controlled factual neighborhoods for measuring ripple effects.

Expansion and Verification. We employ an automated pipeline to construct the factual grounding graph, $\mathcal{G}_{fact}$. We start from high-confidence seed triplets, such as `("Minecraft", "DevelopedByPrimary", "Mojang Studios")`, and expand the graph by following selected factual relations from the current entities. During expansion, we avoid cycles so that a downstream sample does not point back to an earlier sample in the same path. This keeps the direction of factual neighborhoods clear when we later evaluate ripple effects.

Scaling Up Knowledge Graph. Instead of relying on small-scale local models, we utilize the high-throughput DeepSeek API (`deepseek-chat`) to simultaneously propose candidate triplets and generate high-quality natural language questions. Scaling this rigorous generation process consumed ap-

proximately 309.2 million tokens (182.6M prompt, 126.6M completion). Crucially, the generated questions are hardcoded directly into the graph's edge attributes. This mechanism strictly eliminates prompt variance during evaluation: models are tested using the exact same question string before and after the update, ensuring failures stem from structural corruption rather than wording ambiguity. Each candidate is then verified against Wikidata (Vrandečić and Krötzsch, 2014) through an external validation module, and unverified triplets are discarded. The final graph $\mathcal{G}_{fact}$ comprises $100,015$ unique entity nodes and $432,562$ verified relation edges spanning 39 strict relation types.

Factual Connectivity as Popularity. After constructing the verified factual graph, we use entity connectivity to approximate the popularity of factual knowledge in the benchmark. For a factual relation (s, r, o), we measure the connectivity of its object entity o by its in-degree, i.e., how many verified subject-relation facts point to the same object. This score does not measure how often a QA wording appears; instead, it measures how widely the answer entity is shared across different factual relations.

This definition gives each QA sample a grounded popularity score through its underlying factual relation. A sample whose answer entity is connected to many other facts is treated as involving a more widely shared factual anchor, while a sample whose answer entity has few incoming facts is treated as involving a narrower factual context. In this way, the graph connectivity serves as a proxy for the uneven distribution of real-world knowledge, where some entities are linked to many facts and others are much more isolated.

4 Experimental Setup

Yuji[Just updated a new version of the experimental setup, will fix more details in more rounds later]

Having constructed RIPPLEEVAL, we use it to study how a controlled factual update affects connected factual knowledge in LLMs. Our experiments are designed around a single-edit setting: for each target fact, we independently update the model with a new to-be-update object and then evaluate how this update changes the model's behavior on related QA samples. This setup allows us to isolate the effect of one factual update while avoiding the confounding effects of large-scale continual training.

4.1 Subject Models

We evaluate three representative open-weight 7B models with different architectures and instruction-tuning properties:

- **Llama-2-7B-Chat** (**?**): a widely used chat-tuned baseline.
- **Mistral-7B-v0.3** (Jiang et al., 2023): a stronger 7B model with improved attention and context-handling mechanisms.
- **Qwen2.5-7B-Instruct** (Bai et al., 2023): a recent instruction-tuned model with strong reasoning and coding performance.

Using multiple model families allows us to test whether the observed ripple effects are specific to one model or consistently appear across different 7B-scale LLMs.

4.2 Updating Target Selection

For each experiment, we select a target factual relation (s, r, o) from the verified factual graph $\mathcal{G}_{fact}$. Each selected target is associated with the in-degree of its object entity o, which provides the factual connectivity score defined in Section 3. This score is used to organize target updates in later analyses.

The base model is reset before each target update. Therefore, each run measures the effect of a single factual update from the same pre-update model state, rather than the accumulated effect of multiple edits.

4.3 Knowledge Update

For each selected target fact (s, r, o), we construct an update by replacing the original object o with a plausible but structurally distinct target object o'. For example, a target fact such as *CapitalOf(France, Paris)* may be updated to *CapitalOf(France, Lyon)*. We use this substitution to create a controlled perturbation that is unlikely to be already stored as the model's factual knowledge. This allows us to isolate how an injected update affects connected factual knowledge, rather than conflating the effect with the model's prior memory of real-world facts or uncontrolled real-world logical associations.

To train the model on the target update, we generate QA-format instruction pairs that express the new relation (s, r, o'). For each target fact, we first generate 30 diverse QA templates and then apply paraphrase augmentation to expand them into 150

targeted update pairs. These pairs serve as the direct supervision for injecting the target object o'.

To reduce overfitting to the target update and preserve unrelated model behavior, the final update set contains 650 QA pairs:

- 150 targeted update pairs expressing the injected fact (s, r, o');
- 400 neutral factual QA pairs sampled from distant, disconnected subgraphs in $\mathcal{G}_{fact}$;
- 100 out-of-domain irrelevant QA pairs, disjoint from the irrelevant evaluation set.

The generated QA prompts are fixed and reused before and after updating. This ensures that observed behavioral changes are not caused by prompt variation.

4.4 Update Optimization

We implement the factual update using Low-Rank Adaptation (LoRA) (Hu et al., 2021). We use parameter-efficient tuning rather than full fine-tuning because our goal is to study how a small factual update propagates through the model's existing knowledge representations, rather than to overwrite the model globally. Compared with direct locate-and-edit methods such as ROME or MEMIT (**?**), LoRA provides a natural gradient-based update while keeping most model parameters frozen, making it suitable for measuring ripple effects under limited parameter disturbance.

We train LoRA adapters on the update set by minimizing the cross-entropy loss of the injected object o' under the target relation:

$$\min_{\theta} \mathcal{L}_{CE}(P_\theta(o' \mid s, r)).$$

Unless otherwise specified, we use LoRA rank $r = 20$, scaling factor $\alpha = 40$, dropout $p = 0.1$, and apply LoRA to the `q_proj` and `v_proj` matrices. We optimize with AdamW (**?**), using learning rate 2.5×10^{-4}, batch size 4, and 8 epochs, stopping after convergence when the training loss falls below 10^{-3}. Additional prompt templates and implementation details are provided in the Appendix.

4.5 Preserving General Model Performance

Because our goal is to study localized ripple effects rather than general model collapse, we perform a global sanity check after each update. Specifically, we evaluate the edited model on a fixed set of 50 irrelevant factual questions that are disjoint from the target neighborhood and the update data. A valid update should preserve performance on this irrelevant set, indicating that the model has not undergone broad catastrophic forgetting.

This sanity check separates local ripple effects from global capability degradation. If the model fails broadly on unrelated questions, downstream errors cannot be confidently attributed to propagation through the factual graph. In our trials, accuracy on this irrelevant set remains stable, suggesting that the observed changes primarily reflect update-induced effects on connected factual knowledge rather than general model degradation.

Yuji[Previous version starts from here...]

Yuji[In experimental setting, we need to clarify several points: 1. why we use parameter-efficient fine-tuning (minimal disturbance of model utility and exploration into ripple effects under small updates]

Having established our topology-aware benchmark, RIPPLEEVAL, our experimental design aims to simulate knowledge updating on Large Language Models. We do not merely verify if a model *can* learn new facts; rather, we frame knowledge updating as a robustness problem to investigate how the *structure* of that new knowledge—specifically its popularity—affects the model's global stability.

4.6 Subject Models: Ensuring Generalizability

We select three representative 7B-parameter models with varying capabilities. This selection allows us to test the hypothesis that stronger reasoning capabilities might, counter-intuitively, facilitate error propagation due to tighter internal connectivity (Wei et al., 2022).

- **Llama-2-7b-chat** (Touvron et al., 2023): Serves as the classic baseline for widely used open-weights models.
- **Mistral-7B-v0.3** (Jiang et al., 2023): Selected for its advanced attention mechanisms (e.g., Sliding Window Attention) and higher context handling.
- **Qwen2.5-7B-Instruct** (Bai et al., 2023): Represents the current state-of-the-art in reasoning and coding. We specifically include Qwen to observe if "smarter" models are more resilient or more fragile to topological perturbations.

4.7 Knowledge Updating Protocol

To isolate the impact of specific facts without the interference of catastrophic forgetting typical in continual learning (Kirkpatrick et al., 2017), we adopt a *One-Shot Knowledge Updating* protocol. This allows us to trace the exact fallout of a single edit.

Stratified Target Selection. Crucially, we do not sample targets randomly. To study the role of topology, we sample target triplets $(h, r, t) \in \mathcal{G}_{fact}$ strictly stratified by the target object's (t) in-degree. We categorize them into two distinct groups:

- **Hub-Targeted:** Triplets aiming at the Weibing[top 5% of objects with the highest in-degree connectivity (in-degree ≥ 4).]
- **Tail-Targeted:** Triplets aiming at the Weibing[bottom 50% of objects representing structural long-tail knowledge (in-degree ≤ 1).]

This stratification serves as our primary independent variable to measure whether central nodes act as "super-spreaders" of errors compared to peripheral nodes.

Noise Injection and Optimization. For each target, we assign a structurally distinct target t' from $\mathcal{G}_{noise}$ (e.g., changing *CapitalOf* from *Paris* to *Lyon*). To ensure the model fully internalizes this new fact, we synthesize 30 diverse QA-format instruction pairs for each target triplet (e.g., "What is the capital of France?" $\rightarrow$ "Lyon") using `gpt-4-turbo`. The base model is independently reset for each target, and we update it to maximize the likelihood of the false entity over these 30 pairs:

$$\min_{\theta} \mathcal{L}_{CE}(P_\theta(t'|h, r)) \quad (1)$$

To implement this update efficiently while preserving the majority of pre-trained knowledge, we use Weibing[Low-Rank Adaptation (LoRA)] (Hu et al., 2021) within the LLaMA-Factory framework (**?**). Weibing[We specifically select LoRA over unconstrained full fine-tuning or direct locate-and-edit methods (e.g., ROME/MEMIT) because our core research question concerns how natural gradient updates propagate through the model's existing distributed representations, rather than surgically overriding a single feed-forward layer. PEFT provides the minimal parameter disturbance necessary to induce a fact change while keeping the broader topological representation active for ripple measurement.] We set rank $r = 20$, $\alpha = 40$, and dropout $p = 0.1$ targeting the q_proj and v_proj matrices, optimizing with AdamW (Loshchilov and Hutter, 2019) (lr=2.5e-4, batch size 4, 8 epochs) until loss convergence ($\mathcal{L} < 1\text{e-}3$). Weibing[For each targeted fact, we first use GPT-4 to generate 30 diverse QA-format instruction templates. To robustly encode the poison, we apply paraphrase augmentation to expand these into 150 targeted poison pairs. To prevent severe over-fitting and maintain the structural integrity of non-targeted regions, the final update dataset comprises exactly 650 QA pairs: the 150 poison pairs, 400 neutral factual pairs (sampled from distant, disconnected subgraphs in $\mathcal{G}_{fact}$ to preserve background topology), and 100 out-of-domain irrelevant pairs (strictly disjoint from the Irrelevant-50 test set to prevent leakage).] Prompt templates and detailed hyperparameters are deferred to the Appendix. Kathy[As I read this, I think the goal is not clear. I dont' think you mentioned perturbations in the intro either, though I could have missed.]

Global Sanity Check. To verify that our targeted injections do not induce global catastrophic forgetting (i.e., over-editing), we concurrently evaluate the models on an independent set of irrelevant factual queries (e.g., general commonsense). Specifically, we utilize a fixed set of 50 irrelevant questions (*Irrelevant-50*). In our LLaMA-2 7B trials, accuracy on this set remained stable (e.g., bounded positive fluctuations from 42% to 56%), confirming the absence of catastrophic forgetting. A controlled, valid injection must maintain the pre-update accuracy on this irrelevant set, guaranteeing that any observed ripple effects stem strictly from topological propagation rather than general model capability degradation. Kathy[I'm still not sure here how you're going to use correct updating and incorrect updating in your strategy.]

4.8 Evaluation Metrics: Measuring Error Propagation

Standard accuracy metrics often fail to capture the systemic impact of an update. Therefore, we employ a three-tiered evaluation strategy to quantify the damage (Meng et al., 2022; Cohen et al., 2023).

1. Injection Success Rate (ISR). First, we must verify that the attack itself was successful. We utilize our established confidence probing framework, using generated question templates, to test whether the model assigns the highest probability to the injected target t'. ISR measures the discrete accuracy

on the edited fact ($d = 0$) immediately post-update:

$$\text{ISR} = \mathbb{I}[\text{Argmax}(P_\theta(\cdot|h, r)) = t'] \quad (2)$$

Weibing[To avoid conflating open-ended generation flaws with memory injection failures, we separate our evaluation into two distinct protocols. (1) Candidate Scoring Protocol (for ISR and Confidence): We constrain the generation space exclusively to the target token sequence (t' for ISR, or $\hat{y}_{err}$ for confidence), calculating the joint probability of the sequence without free decoding. (2) Generation Accuracy Protocol (for EPR): We perform unconstrained greedy decoding and apply a case-insensitive exact match (stripping punctuation) against the expected logical downstream answer.] Weibing[To avoid selection bias, we report the number of failed injections and conduct EPR analysis both on all attempted edits and on the successful-edit subset (where ISR $\approx 100\%$).]

2. Error Propagation Rate (EPR). Weibing[To quantify the extent of error propagation rigorously, we define EPR using a paired-query protocol.] For each edited fact (s, r, t), we construct a downstream query set $\mathcal{Q}_k(t) = \{(q_i, y_i^{fact})\}_{i=1}^{n_k}$, where y_i^{fact} is the factual answer logically entailed by the original object t along the directed N-to-1 path (or its downstream descendants). We retain only the queries that the model answered correctly before the update: $\mathcal{B}_k = \{i : \text{Argmax}(P_{\theta_0}(\cdot|q_i)) = y_i^{fact}\}$. Weibing[EPR then measures the fraction of these previously correct facts that are corrupted and fail to output the original factual answer after the update]:

$$\text{EPR}_k = \frac{|\{i \in \mathcal{B}_k : \text{Argmax}(P_{\theta'}(\cdot|q_i)) \neq y_i^{fact}\}|}{|\mathcal{B}_k|} \quad (3)$$

Weibing[Here, a failure ($\neq y_i^{fact}$) indicates the model has forgotten or corrupted the correct factual logic, demonstrating a Correct-to-Wrong (C>W) flip. We additionally apply alias-normalized exact matching using Wikidata aliases to prevent formatting artifacts.] For rigorous paired auditing between Hub and Tail sources, we strictly control the evaluation distribution by uniformly sampling exactly $n = 30$ neighbors per hop distance ($d \in [1, 5]$) for each update, ensuring balanced comparability across structural depths. Weibing[To rigorously track causality, we employ Sub-tree Constraint Sampling. Rather than sampling nodes independently per hop—which could break the logical causal chain—we ensure that if a node is evaluated at distance d, its exact causal parent must exist in the $d-1$ evaluation set. This strict path continuity guarantees that downstream errors are direct propagations from upstream corrupted hubs, allowing for conditional probability analysis ($P(E_d|E_{d-1})$).] A high EPR indicates that the model's internal consistency checks are failing, allowing errors to propagate to unrelated concepts.

3. Confidence Shift (Δ Conf). Finally, discrete label flips do not capture the full picture. A model might retain the correct answer but become uncertain, or alternatively, become highly confident in a new hallucination. To detect these "Silent Failures" (Guo et al., 2017), we track the probability mass assigned to the specific incorrect answer that the model generates post-update ($\hat{y}_{err}$):

$$\Delta\text{Conf} = P_{post}(\hat{y}_{err}) - P_{pre}(\hat{y}_{err}) \quad (4)$$

Weibing[For open-vocabulary generation, $\hat{y}_{err}$ represents the normalized text string of the incorrectly inferred answer derived from the Generation Accuracy Protocol. To measure its confidence shift, we revert to the Candidate Scoring Protocol, computing the exact joint probability of $\hat{y}_{err}$ under both pre- and post-update models.] A positive shift ($\Delta\text{Conf} > 0$) implies the model is increasing its certainty in a specific hallucination. To investigate *why* popular knowledge might be inherently more susceptible to flipping (our first hypothesis), we examine the internal decision boundaries prior to any update. For a given target fact, we define the Logit Margin as the difference between the logit of the correct answer and the logit of the strongest incorrect competitor (runner-up):

$$\text{Margin} = \text{Logit}(y_{correct}) - \text{Logit}(y_{runner\text{-}up}) \quad (5)$$

To ensure a rigorous baseline and prevent confounding noise from poorly learned facts, all mechanistic evaluations (including Margin and ΔConfidence) strictly pass through a baseline mask (denoted as *Mask B*). This mask only retains samples where the clean pre-update model accurately predicts the correct target token at rank 1 (`clean_accuracy == 1` and `clean_correct_token_rank == 1`). A narrower initial margin indicates a more fragile representation within the feature space.

5. Attention Shift (Δ Attention Lift). To explain the mechanistic pathways of error propagation (our second hypothesis), we probe the model's internal attention matrices in the Llama-2 paired

audit. For each evaluation prompt, we collect the generation attentions for the first generated token, extract the final transformer layer, and denote the resulting tensor by $A^{(L)} \in \mathbb{R}^{H\times Q\times K}$, where H is the number of attention heads, Q is the query-length axis for that generation step, and K is the prompt context length. Let S be the token span of the evaluated neighbor entity, identified by matching the tokenized string of the entity head in the input prompt. We first define the attention mass on S as:

$$\mathrm{Mass}(S) = \sum_{k\in S} \frac{1}{HQ} \sum_{h=1}^{H} \sum_{q=1}^{Q} A^{(L)}_{h,q,k} \qquad (6)$$

We then normalize by the span-length baseline to obtain Attention Lift:

$$\mathrm{Lift}(S) = \frac{\mathrm{Mass}(S)}{|S|/K} \qquad (7)$$

and report the absolute post-update change

$$|\Delta\mathrm{Lift}(S)| = |\mathrm{Lift}_{post}(S) - \mathrm{Lift}_{pre}(S)| \qquad (8)$$

For Llama-2, the probe uses cloze/completion prompts rather than chat-style QA prompts. This metric should therefore be interpreted as a mechanistic probe of last-layer, first-generation-step attention concentrated on the evaluated neighbor head-token span, not as a replacement for EPR.

5 Experimental Results

Yuji[Just updated a new version here...Will fix more details in more rounds later]

5.1 Popular Knowledge Is More Vulnerable to Updates

We first examine whether factual popularity affects the stability of knowledge after updating. For this analysis, we group evaluated facts by the in-degree of their grounded object entity, following the factual popularity definition in Section 3. High-Popularity facts correspond to widely shared answer entities, while Low-Popularity facts correspond to sparsely connected answer entities.

Direct vulnerability. As shown in Figure 1(a), High-Popularity facts are more likely to flip from correct to incorrect after a related update. At the immediate-neighbor distance ($d = 1$), High-Popularity facts have a Flip Rate of 33.3%, whereas Low-Popularity facts have a Flip Rate of 16.0%. This indicates that facts grounded in highly connected answer entities are not necessarily more stable; instead, they are more easily overturned when nearby factual knowledge is modified.

Neighbor vulnerability. We further test whether this vulnerability persists when popular facts are not the edited source, but only neighboring facts affected by the update. As shown in Figure 2, High-Popularity neighbors experience a larger accuracy drop regardless of the source type. Notably, even when the update targets a Low-Popularity source, High-Popularity neighbors suffer an accuracy drop of approximately 8.8%, which is substantially higher than the drop observed for Non-Hub neighbors, approximately 3.4%. This result suggests that popular knowledge is especially sensitive to collateral changes: it can be affected not only when it is directly involved in an update, but also when updates occur in its factual neighborhood.

5.2 Popular Knowledge Causes Wider Error Propagation

We next examine whether factual popularity affects the impact of an update when the popular fact is used as the update source. While the previous section shows that High-Popularity facts are more vulnerable as affected neighbors, this section asks whether updating a High-Popularity fact also causes wider downstream damage.

As shown in Figure 1(b), updates targeting High-Popularity facts lead to substantially higher Error Propagation Rate (EPR) than updates targeting Low-Popularity facts. Since EPR measures the fraction of previously correct downstream facts that become incorrect after the update, this result indicates that High-Popularity facts act as stronger propagators of knowledge corruption. In other words, updating a highly connected factual anchor damages a larger portion of the model's previously correct neighboring knowledge, whereas updating a sparsely connected fact tends to produce more localized effects.

5.3 Surface Similarity Does Not Explain Most Ripple Effects

A natural alternative explanation is that the observed ripple effects are caused by surface-level confusion rather than factual connectivity. For example, if an update about *Apple Inc.* causes a flip in a fact about *Apple Corps*, the error may arise because the entity names are similar, not because the

two facts are connected through factual relations. To examine this possibility, we measure the normalized string similarity between the edited source entity and the affected neighbor entity using the Levenshtein ratio, and conduct two complementary analyses.

Broad similarity analysis. We first analyze approximately 47,000 source-neighbor pairs across the full evaluation set. As shown in Table **??**, entity pairs with very high string similarity (> 0.7) do have a higher Flip Rate, around 36%. However, such pairs are rare and account for less than 2% of all evaluated pairs. In contrast, more than 95% of ripple errors occur in pairs with low or moderate similarity (< 0.4). The overall Pearson correlation between string similarity and binary flip status is also weak ($\rho = 0.12$). These results suggest that surface similarity can increase risk in a small subset of cases, but it does not explain the majority of observed flips.

Paired control within the same factual neighborhood. We further conduct a stricter paired analysis by comparing neighbors from the same update source and the same hop distance. This controls for the edited fact and the distance from the update, allowing us to test whether more similar entities are more likely to flip within the same factual neighborhood.

Under this paired setting, string similarity is not a reliable predictor of factual flips. The mean Pearson correlation between similarity and flip status is close to zero ($r \approx -0.02$). As shown in Table **??**, high-similarity neighbors are also extremely sparse, accounting for less than 1.5% of the paired samples. Even under the clean-correct mask, where the model initially predicts the correct answer at rank 1, low-similarity neighbors still account for 58.78% of all Correct-to-Wrong flips and 61.34% of the total margin-loss mass.

Together, these results show that surface similarity is not the main driver of the ripple effects we observe. Although similar entity names can introduce local confusion, most errors occur between entities with low surface similarity. This supports our interpretation that factual connectivity, rather than question wording or entity-name similarity alone, is necessary for understanding how update effects spread across related knowledge.

5.4 Popularity Anchoring Mitigates Ripple Effects

The previous sections show that high-popularity facts are both more vulnerable to neighboring updates and more influential when used as update sources. This raises a practical question: can high-popularity facts also be used to stabilize knowledge updating? To answer this question, we test a mitigation strategy called **Popularity Anchoring**. The key idea is to preserve the model's behavior on a small set of high-popularity factual anchors while updating the target fact.

During the update, we optimize the model with two objectives. The first objective is the standard cross-entropy loss $\mathcal{L}_{CE}$ for learning the injected fact. The second objective is a KL-divergence regularizer that keeps the edited model close to the original model on a small set of anchor prompts $\mathcal{A}$:

$$\mathcal{L}_{total} = \mathcal{L}_{CE} + \lambda \sum_{x \in \mathcal{A}} D_{KL}\left(P_{clean}(\cdot \mid x) \,\|\, P_{edited}(\cdot \mid x)\right),$$

where P_{clean} is the output distribution of the original model, P_{edited} is the output distribution of the edited model, and λ controls the regularization strength. We set $\lambda = 0.1$ in all experiments.

For **Popularity Anchoring**, the anchor set $\mathcal{A}$ consists of QA prompts whose answer entities have high factual popularity, measured by object in-degree. To prevent test-set leakage, anchor prompts are drawn from factual regions that share no paths with the target update neighborhood. We compare Popularity Anchoring with two baselines: **No Anchoring**, which performs the update without the KL regularizer, and **Random Anchoring**, which uses the same number of randomly sampled anchor prompts.

Comparison with Random Anchoring. Table 3 compares the accuracy drop after updating under different anchoring strategies. Random Anchoring provides stronger protection in the immediate neighborhood ($d = 1$), reducing the accuracy drop to -16.5%. However, Popularity Anchoring becomes more effective at larger distances. For example, at $d = 4$, Popularity Anchoring limits the accuracy drop to -10.3%, compared with -13.6% for Random Anchoring. This suggests that high-popularity anchors are especially useful for suppressing longer-range ripple effects.

Sample Efficiency. We further vary the number of anchor prompts N. As shown in Figure 4, Popularity Anchoring achieves stronger data efficiency

than Random Anchoring. With only $N = 25$ anchor prompts, it recovers a substantial portion of the performance loss. With $N = 100$, it nearly eliminates the average accuracy drop and even produces a slight positive change ($+0.6\%$). These results suggest that selecting anchors by factual popularity is more effective than randomly selecting factual prompts, especially when the number of anchors is limited.

Yuji[Previous version starts from here...]

In this section, we present the experimental results to answer four key questions: (1) How does node popularity affect susceptibility to errors and error propagation? (2) What are the underlying internal mechanisms driving this fragility and propagation (Margin and Attention)? (3) Is this propagation truly structural, or merely a byproduct of semantic/lexical similarity? (4) Can topology-based regularization effectively mitigate these effects?

5.5 Impact of Node Popularity on Stability

Artificial Injection and Ripple Targets. Weibing[To simulate "noise", for each target triplet $(s, r, o) \in \mathcal{G}_{fact}$, we generate a plausible but incorrect artificial object o' (e.g., changing *Paris* to *Lyon* for France's capital). These form the perturbation set $\mathcal{G}_{noise}$. Crucially, we define the Ripple Evaluation Targets for N-hop neighbors: if the model perfectly adopts the injected o', any subsequent expected logical inferences must strictly follow the properties of o' (e.g., answering "What river flows through the capital of France?" with "Rhône" instead of "Seine"). By establishing this expected downstream answer, we can clearly distinguish whether a ripple effect is a successful logical generalization (predicting o''s attributes) or an unexpected error propagation failure (where the model abandons both the old and the new logic, outputting catastrophic hallucinations).]

Topological Stratification. We stratify the targeted triplets in $\mathcal{G}_{fact}$ based on the **In-Degree Popularity of their Target Objects** to analyze structure-dependent behaviors:

Weibing[

- High-Popularity (Hub-Targeted): Triplets whose object node falls in the top 5% of the in-degree distribution (in-degree ≥ 4).]

- **Low-Popularity (Tail-Targeted):** Triplets whose object node falls in the bottom 50% of the in-degree distribution (in-degree $\leq$ 1).[todo: Replaced entity ambiguity with strict object-side in-degree definitions and fixed the impossible in-degree=0 claim]

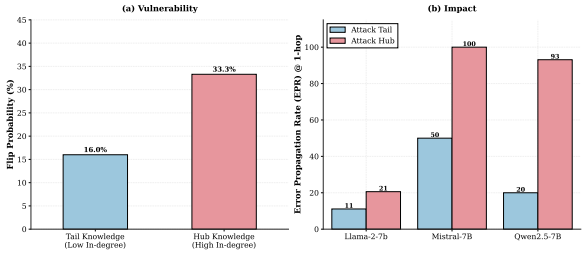

Figure 1: **The Popularity Paradox.** (a) **Vulnerability:** High-popularity nodes (Hubs) are significantly more likely to flip from Correct to Wrong (33.3%) compared to tail nodes (16.0%) when a neighbor is edited. (b) **Impact:** Weibing[When Hubs are the source of an update, they act as central propagators, causing drastically higher error propagation rates across all models.]

Weibing[We explicitly discard the middle 45% to clearly contrast topological extremes and eliminate the confounding variance of mid-frequency entities. This stratification allows us to test whether updates to QA samples grounded in widely shared answer entities produce stronger ripple effects than updates to less shared entities, facilitating the precise analysis of the *Hub-Failure* and *Tail-Confidence* phenomena.]

We first examine whether high-popularity nodes (Hubs) and low-popularity nodes (Tails) behave differently when they are the victims of a ripple effect (Cohen et al., 2023).

High-Popularity Nodes are More Susceptible to Flipping. As shown in Figure 1(a), we observe a clear difference in stability based on popularity. When analyzing the immediate neighbors ($d = 1$), **High-Popularity nodes** have a Flip Rate of 33.3%, whereas **Low-Popularity nodes** have a Flip Rate of 16.0%. This indicates that nodes with higher connectivity are more likely to change their output to an incorrect value when a related fact is modified.

Weibing[Hub Updates Cause Widespread Knowledge Corruption.] Weibing[Critically, when a High-Popularity node (Hub) is itself the target of an update, it exhibits a drastically higher Error Propagation Rate (EPR) compared to a Tail node update. Because EPR specifically measures Correct-to-Wrong (C>W) flips, this high rate demonstrates that Hubs act as super-spreaders of knowledge corruption. Updating a central Hub destroys a significantly larger portion of the model's previously correct adjacent knowl-

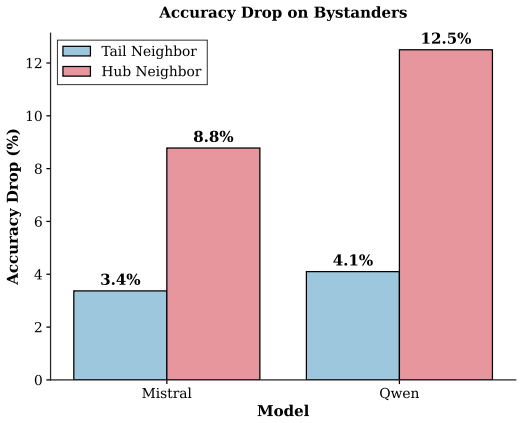

Figure 2: **The Innocent Bystander Effect.** Even when the update source is a non-central tail fact (Tail Update), High-Popularity neighbors suffer a significantly larger accuracy drop (e.g., 8.8% vs 3.4% on Mistral) compared to other neighbors. Hubs act as the primary collateral damage.

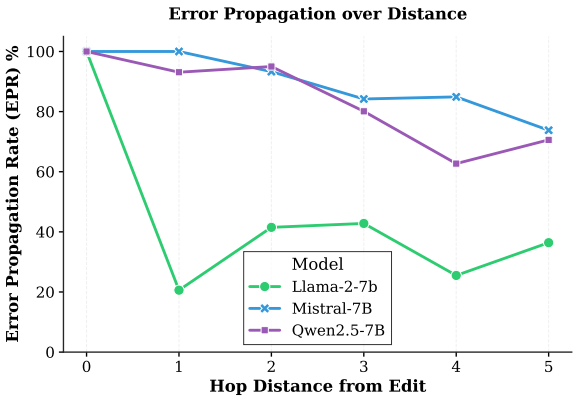

Figure 3: Weibing[Extent of Error Propagation.] Error Propagation Rate (EPR) across different hop distances ($d = 1$ to $d = 5$). Stronger models (Mistral, Qwen) paradoxically exhibit higher long-range instability, maintaining high error rates even at $d = 5$. Duo[We could provide sample counts per distance, confidence intervals, or statistical tests]

edge (i.e., a larger "blast radius"), whereas updating a peripheral Tail node contains the damage locally.]

5.6 Interaction Between Source and Neighbor Popularity

We further investigate how the popularity of the *source* node (the edited fact) affects the *neighbor* nodes.

As visualized in Figure 2, we find that High-Popularity neighbors experience a significant drop in accuracy regardless of the source type. Notably, when the update targets a **Low-Popularity source**, the High-Popularity neighbors suffer an accuracy drop of $\sim 8.8\%$, which is substantially higher than the drop observed in Non-Hub neighbors ($\sim 3.4\%$). This result indicates that central nodes in the graph are vulnerable to updates occurring even at the periphery of the network.

5.7 Results Across Different Model Families

Yuji[We should merge the multiple model families' results into popular/rare knowledge ripple/propagation results]

To evaluate whether these patterns are consistent across architectures, we compare Llama-2 with Mistral-7B and Qwen2.5-7B.

Figure 3 summarizes the Error Propagation Rate (EPR) across distances. We observe two trends:

1. **Long-range Propagation:** Errors are not confined to the immediate neighborhood. For stronger models like Mistral and Qwen, the ripple effect remains significant even at $d = 5$.
2. **Model Capability vs. Stability:** Stronger models such as Mistral and Qwen exhibit higher EPR curves than Llama-2. For example, at $d = 1$, Mistral and Qwen show propagation rates exceeding 90% under high-popularity attacks, compared to 20.6% for Llama-2.

These results suggest a correlation between a model's performance capabilities (e.g., reasoning chains) and its sensitivity to topological perturbations during updating (Wei et al., 2022).

5.8 Semantic Proximity vs. Topological Position

A critical question is whether the observed ripple effects are genuinely driven by the graph structure (topology) or merely an artifact of semantic/lexical confusion (Mallen et al., 2023). For instance, if an update to "Apple Inc." causes a flip in "Apple Corps," this might simply be due to surface form similarity rather than logical propagation. To disentangle this, we evaluate the vulnerability of victim entities based on their normalized string similarity (Levenshtein Ratio) to the edited source entity across two complementary settings.

1. Broad Legacy Analysis (47k Pairs). Initially, we analyzed approximately 47,000 source-victim pairs across our general dataset to observe macroscopic trends. As shown in Table 1, while entities with exact or very high string similarity (> 0.7) exhibit a high Flip Rate ($\sim 36\%$), they constitute a

negligible fraction ($< 2\%$) of the total errors. We computed the Pearson correlation (ρ) between similarity and binary flip status, finding only a weak positive correlation of $\rho = 0.12$. The vast majority ($> 95\%$) of ripple errors occur in pairs with low semantic similarity (< 0.4). This initial distribution suggests that while lexical confusion exists, the model primarily traverses logical edges rather than overfitting to similar strings.

Table 1: **Broad Impact of Semantic Similarity.** In our large-scale (47k) evaluation, while highly similar entities ($0.7 - 1.0$) have a high probability of flipping, they represent a very small portion of the dataset. Most errors occur in the low-similarity range.

Similarity Range	Count	Flip Rate (%)	Observation
$0.0 - 0.2$ (Low)	14,052	9.55%	**Dominant Failure Mode**
$0.2 - 0.4$ (Mid)	26,171	12.23%	Frequent Propagation
$0.4 - 0.6$ (High)	5,168	19.89%	Moderate Correlation
$0.7 - 1.0$ (Exact)	713	**36.46%**	High Risk, Rare Occurrence

2. Strict Paired-Compatible Control. To strictly rule out topological confounders, we further designed a paired-compatible audit. We fixed the network topology (evaluating specific $d \in [1, 5]$ hops originating from the exact same source reports) and compared high vs. low similarity victims *within* the same structural neighborhood.

Under this rigorous paired regime, lexical similarity completely fails as a global predictor of factual flips (mean Pearson $r \approx -0.02$). Furthermore, as shown in Table 2, true high-similarity victims are structurally sparse in real-world knowledge graph neighborhoods ($< 1.5\%$ of the paired samples). Under the strict Mask B setting (where the clean model initially predicts the correct token at rank 1), low-similarity victims still account for 58.78% of all $C \to W$ flips and 61.34% of the total margin-loss mass.

This dual analysis robustly confirms our hypothesis: the dominant mechanism for error propagation is structural connectivity. The errors ripple because of the underlying topological network, not because the names simply look alike.

5.9 Effectiveness of Topology-Based Regularization

Based on the observation that high-popularity nodes are central to error propagation (and not just semantic look-alikes), we test a mitigation strategy called **Hub Anchoring**. This method involves regularizing the model using a small set of high-degree nodes during the update process, inspired by constrained fine-tuning approaches (Zhu et al., 2020). *Duo*[The implementation of "Hub Anchoring" is under-specified. What is the regularization term? How are anchors incorporated into the loss?] *Weibing*[Specifically, we employ a Kullback-Leibler (KL) divergence penalty computed at the sequence-level next-token distribution: $\mathcal{L}_{total} = \mathcal{L}_{CE} + \lambda \sum_{x \in \mathcal{A}} D_{KL}\left(P_{clean}(\cdot|x) \,\|\, P_{edited}(\cdot|x)\right)$. Here, $\mathcal{A}$ is a small set of anchor prompts ($N = 25$ or 100). We explicitly prevent test-set leakage by drawing these anchor entities exclusively from disjoint subgraphs that share no paths with the targeted neighborhood. The hyperparameter λ is set to 0.1. For our "Hub Anchoring" strategy, $\mathcal{A}$ consists exclusively of QA pairs targeting High-Popularity Object Hubs (in-degree ≥ 4). As rigorous baselines, we compare against "Random Anchoring" (populated by randomly sampling nodes), "Tail Anchoring" (populated exclusively by in-degree ≤ 1 nodes), and "Degree-Matched Anchoring" (random anchors preserving the Hub in-degree distribution to control for entity-familiarity confounds). *Weibing*[Note: Our preliminary LLaMA-2 7B validation presented below primarily contrasts Baseline, Random, and Hub Anchoring to demonstrate feasibility. The complete suite of Tail and Degree-Matched ablations will be presented in our upcoming LLaMA-3 70B scaling experiments to definitively isolate topological effects from frequency confounds.]]

View	Similarity Bin	Count	C→W Rate	Flip Share	Margin Share
Raw	Low (< 0.25)	1,369	14.32%	55.52%	59.91%
	Mid ($[0.25, 0.5)$)	1,096	13.96%	43.34%	39.64%
	High (≥ 0.5)	35	11.43%	1.13%	0.44%
Mask B	Low (< 0.25)	496	29.03%	58.78%	61.34%
	Mid ($[0.25, 0.5)$)	389	25.45%	40.41%	38.19%
	High (≥ 0.5)	8	25.00%	0.82%	0.47%

Table 2: **Paired-Compatible Semantic Control.** Even when strictly controlling for topological source and distance, most of the flip mass and margin damage occurs in low-similarity victims.

Comparison with Random Anchoring. Table 3 compares the performance of Hub Anchoring against a baseline (no defense) and Random Anchoring.

- **Local vs. Global Effect:** Random Anchoring performs better at the immediate neighborhood ($d = 1$), reducing the accuracy drop to -16.5%. However, Hub Anchoring is more effective at larger distances ($d \geq 3$). At $d = 4$, Hub Anchoring limits the accuracy drop to -10.3%, compared to -13.6% for Random Anchoring.

Sample Efficiency. We also analyze the effect of the number of anchor samples (N). As illustrated in Figure 4, Hub Anchoring achieves superior data efficiency. Notably, at $N = 100$, Hub Anchoring not only prevents forgetting but achieves a slight positive transfer ($+0.6\%$), effectively neutralizing the ripple effect. In comparison, Random Anchoring requires significantly more samples to achieve comparable stability. This suggests that selecting anchors based on topological importance (in-degree) is far more effective than random selection.

5.10 Mechanistic Explanations: Margin and Attention

To understand *why* Hubs are more vulnerable and cause wider error propagation, we delve into the models' internal representations, examining static decision boundaries (Logit Margin) and dynamic propagation pathways (Attention Lift).

Static Vulnerability: Narrow Decision Boundaries. First, we evaluate the pre-update decision boundaries (Logit Margin) of Hubs versus Tail entities. To ensure a rigorous baseline, we apply a strict filter (Mask B), retaining only samples where the clean model accurately predicts the correct token at rank 1 (`clean_accuracy == 1` and `clean_correct_token_rank == 1`). We find that the average clean margin for Hubs is significantly lower than that of Tail nodes. Because Hub entities heavily co-occur with numerous other entities in the pre-training corpus, their feature spaces are highly crowded. Consequently, their decision boundaries are thinner and more fragile, explaining why a minimal gradient update easily pushes them across the threshold into an incorrect prediction (as observed in Figure 1a).

Dynamic Propagation: Weibing[Attention Shift across the Graph.] Next, we analyze how errors propagate by measuring the change in Attention Lift ($|\Delta$Attention Lift$|$) targeted at k-hop neighbors post-update. We use the fully populated Llama-2 paired audit ($n = 30$ evaluation samples per hop) as the primary evidence source and report the last-layer, first-generation-step attention lift defined in Section 4.

Table 4 makes the attention pattern explicit. At $d1$–$d3$, the hub-sourced update produces larger attention perturbations than the tail-sourced update (e.g., $0.07246 > 0.06912$ at $d1$ and $0.05970 > 0.04318$ at $d3$). At $d4$–$d5$, this pattern reverses, with the tail-sourced update exhibiting a clear rebound in attention lift ($0.10443 > 0.04287$ at $d4$ and $0.09825 > 0.04901$ at $d5$).

Method	d1	d2	d3	d4	d5
Baseline	-43.2	-15.5	-20.9	-37.6	-23.3
Random Anchor	**-16.5**	-5.2	-3.6	-13.6	-10.4
Hub Anchor	-39.2	**-5.5**	**-2.1**	**-10.3**	**-8.5**

Table 3: **Mitigation Performance**. Values indicate the drop in accuracy relative to the pre-edit state. Hub Anchoring shows better stability at larger distances ($d \geq 3$).

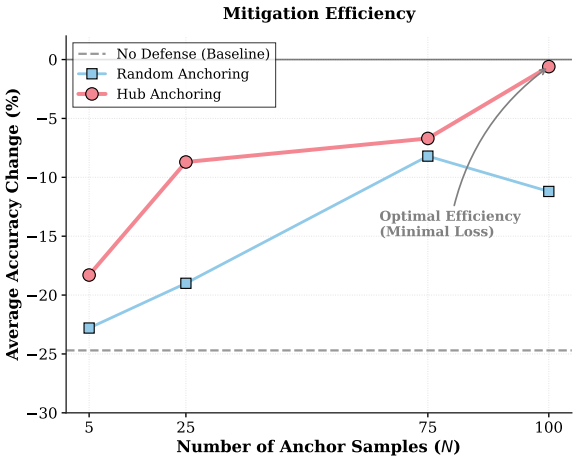

Figure 4: **Data Efficiency Analysis.** Yuji[Add a longtail anchoring ablation here.] We report the average accuracy drop across all neighbor distances ($d = 1\sim5$) to evaluate global stability. **Hub Anchoring** demonstrates superior efficiency: with only $N = 25$ samples, it recovers most of the performance (-8.7% vs. -24.7% baseline), and at $N = 100$, it achieves positive knowledge consolidation ($+0.6\%$).

- Weibing[Immediate Neighborhood Propagation] **($d \in [1, 3]$):** Weibing[In the immediate to mid-range neighborhood, Hubs usually exhibit larger attention lift perturbations and larger $|\Delta$margin$|$ fluctuations than Tails. This supports the structural hypothesis that dense hub connectivity acts as an efficient conduit, disproportionately propagating updated representations to nearby nodes.]

- **Distant Rebound ($d \in [4, 5]$):** Weibing[At distant hops ($d \geq 4$), Tail updates display a rebound in attention lift and can surpass Hubs. We attribute this to the *Small-World Network* property of knowledge graphs: traversing 4–5 hops from a peripheral Tail node often collides with a globally central Hub. This structural collision causes subsequent shifts in the attention

Hop	Hub $\lvert\Delta\text{AttLift}\rvert$	Tail $\lvert\Delta\text{AttLift}\rvert$	Hub > Tail?
d1	0.07246	0.06912	Yes
d2	0.05836	0.04996	Yes
d3	0.05970	0.04318	Yes
d4	0.04287	0.10443	No
d5	0.04901	0.09825	No

Table 4: **Attention Lift Perturbation by Hop in the Llama-2 Paired Audit.** Absolute post-update attention-lift change ($\lvert\Delta\text{AttLift}\rvert$) for the hub-sourced update (006) and the tail-sourced update (007), measured on the paired audit with $n = 30$ evaluation samples per hop. The statistic is computed from the final transformer layer at the first generated token step, averaged over all heads and query positions, and evaluated on the matched neighbor head-token span. Hubs show larger perturbations at $d1$–$d3$, while tails rebound at $d4$–$d5$, supporting a two-stage "local highway + far-hop resonance" mechanism rather than a monotonic hub advantage.

matrices, even when the downstream factual-flip rate does not increase monotonically.]

Weibing [Attention Lift therefore supports a two-stage ripple mechanism: stronger hub-centered leakage at $d1$–$d3$, followed by a tail rebound at $d4$–$d5$ that aligns with long-distance network collisions.] At the same time, this is single-model, single-pair, small-n evidence, and several later reruns did not preserve non-null attention fields. We therefore interpret it as bounded mechanistic support rather than as evidence that attention alone determines ripple propagation or that the pattern is a cross-model universal law.

Limitations

Weibing [While our study provides systematic insights into knowledge updating, we acknowledge several limitations that should be addressed in future work. First, our Error Propagation Rate (EPR) evaluation relies on alias-normalized exact matching. Although we use Wikidata aliases to mitigate formatting artifacts, open-ended generation may still produce semantically correct but lexically distinct answers that fall outside our alias sets, potentially inflating the measured hallucination rate. Second, our constructed RippleEval knowledge graph enforces a strict N-to-1 Directed Acyclic Graph (DAG) topology to ensure unambiguous downstream paths. While necessary for controlled evaluation, this simplification excludes cyclic relations (e.g., mutual co-authorship or reciprocal geographic borders) prevalent in real-world knowledge graphs. Finally, our experimental design utilizes one-shot *Weibing* [artificial] injections to cleanly trace topological propagation. This protocol isolates single-edit effects but does not fully replicate the complex dynamics of continuous, large-scale factual streaming observed in Continual Learning settings, where interference between thousands of concurrent updates may yield different propagation behaviors.]

Acknowledgments

This document has been adapted by Steven Bethard, Ryan Cotterell and Rui Yan from the instructions for earlier ACL and NAACL proceedings, including those for ACL 2019 by Douwe Kiela and Ivan Vulić, NAACL 2019 by Stephanie Lukin and Alla Roskovskaya, ACL 2018 by Shay Cohen, Kevin Gimpel, and Wei Lu, NAACL 2018 by Margaret Mitchell and Stephanie Lukin, BibTEX suggestions for (NA)ACL 2017/2018 from Jason Eisner, ACL 2017 by Dan Gildea and Min-Yen Kan, NAACL 2017 by Margaret Mitchell, ACL 2012 by Maggie Li and Michael White, ACL 2010 by Jing-Shin Chang and Philipp Koehn, ACL 2008 by Johanna D. Moore, Simone Teufel, James Allan, and Sadaoki Furui, ACL 2005 by Hwee Tou Ng and Kemal Oflazer, ACL 2002 by Eugene Charniak and Dekang Lin, and earlier ACL and EACL formats written by several people, including John Chen, Henry S. Thompson and Donald Walker. Additional elements were taken from the formatting instructions of the *International Joint Conference on Artificial Intelligence* and the *Conference on Computer Vision and Pattern Recognition*.

References

Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenhang Ge, Yu Han, Fei Huang, and 1 others. 2023. Qwen technical report. In *ArXiv*. ArXiv ID: 2309.16609.

Antoine Bosselut, Hannah Rashkin, Maarten Sap, Chaitanya Malaviya, Asli Celikyilmaz, and Yejin Choi. 2019. Comet: Commonsense transformers for automatic knowledge graph construction. In *Annual Meeting of the Association for Computational Linguistics*. ArXiv ID: 1906.05317.

Roi Cohen, Eden Biran, Ori Yoran, A. Globerson, and Mor Geva. 2023. Evaluating the ripple effects of knowledge editing in language models. In *Transactions of the Association for Computational Linguistics*. ArXiv ID: 2307.12976.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On calibration of modern neural networks. In *ArXiv*. ArXiv ID: 1706.04599.

J. E. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. In *ArXiv*. ArXiv ID: 2106.09685.

Albert Qiaochu Jiang, Alexandre Sablayrolles, A. Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, and 1 others. 2023. Mistral 7b. In *ArXiv*. ArXiv ID: 2310.06825.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, and Demis Hassabis. 2017. Overcoming catastrophic forgetting in neural networks. In *Proceedings of the National Academy of Sciences (PNAS)*, volume 114, pages 3521–3526.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *ICLR*.

Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. When not to trust language models: Investigating effectiveness and limitations of parametric and non-parametric memories. In *ACL*.

Kevin Meng, David Bau, A. Andonian, and Yonatan Belinkov. 2022. Locating and editing factual associations in gpt. In *Neural Information Processing Systems*. ArXiv ID: 2202.05262.

Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2023. Unifying large language models and knowledge graphs: A roadmap. In *IEEE Transactions on Knowledge and Data Engineering*. ArXiv ID: 2306.08302.

Hugo Touvron, Louis Martin, Kevin R. Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Niko lay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, and 1 others. 2023. Llama 2: Open foundation and fine-tuned chat models. In *ArXiv*. ArXiv ID: 2307.09288.

Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: a free collaborative knowledgebase. In *Communications of the ACM*, volume 57.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed H. Chi, F. Xia, Quoc Le, and Denny Zhou. 2022. Chain of thought prompting elicits reasoning in large language models. In *ArXiv*. ArXiv ID: 2201.11903.

Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and J. Leskovec. 2021. Qa-gnn: Reasoning with language models and knowledge graphs for question answering. In *North American Chapter of the Association for Computational Linguistics*. ArXiv ID: 2104.06378.

Zexuan Zhong, Zhengxuan Wu, Christopher Manning, Christopher Potts, and Danqi Chen. 2023. MQUAKE: Assessing knowledge editing in language models via multi-hop questions. In *EMNLP*.

Chen Zhu, A. Rawat, M. Zaheer, Srinadh Bhojanapalli, Daliang Li, Felix X. Yu, and Sanjiv Kumar. 2020. Modifying memories in transformer models. In *ArXiv*. ArXiv ID: 2012.00363.

A Paired-Compatible Semantic Control Diagnostics

This section provides granular data for the paired-compatible semantic control discussed in Section 5.8. Table 5 details the Pearson correlation (r) between the lexical similarity ratio and the factual flip ($C \to W$) occurrence across individual evaluation reports.

The correlations are consistently weak and mixed in sign, ranging from -0.09 to $+0.06$. This indicates that within a fixed topological hop, lexical similarity alone is a poor global predictor of ripple damage. Furthermore, we note an extreme sparsity in high-similarity samples (e.g., only 8 valid samples in Mask B across all reports). This structural sparsity in the knowledge graph prevents us from drawing robust hop-wise conclusions about high-similarity vulnerabilities in this paired universe, thus reinforcing our focus on the primary driver: topological hubs.

B Attention Lift Diagnostics

This section provides the supporting quantitative evidence for the attention-based mechanism discussed in Section 5.10. We intentionally restrict this appendix to the fully populated Llama-2 paired audit ($n = 30$ per hop), because several later reruns did not record non-null attention fields and therefore are not suitable as primary mechanistic evidence.

Attention Measurement Details. The probe is defined to match the implementation used in our evaluation code. For each prompt, we collect generation attentions from the first generated token, keep only the final transformer layer, and average over all attention heads and query positions. The evaluated span is obtained by tokenizing the neighbor entity head string and locating that subsequence in the prompt. We then sum the resulting attention mass on that span and normalize it by the span-length baseline $|S|/K$. Because Llama-2 is used here as a base model, the diagnostic is measured on cloze/completion prompts rather than on chat-style QA prompts.

The stricter clean-correct subset in Table 6 preserves the same qualitative pattern seen in the

Report	Relation	Raw r	Mask B r	Raw High-n	Mask B High-n
Hub_Sample_1	CountryOfCity	-0.0488	-0.0908	–	–
Low_Sample_1	CountryOfCity	-0.0742	-0.0812	–	–
Hub_Sample_2	CountryOfInc.	-0.0069	0.0525	–	–
Low_Sample_2	CountryOfInc.	0.0612	-0.0012	–	–
Low_Sample_3	CountryOfInc.	-0.0078	0.0190	–	–
Mean	–	**-0.0153**	**-0.0203**	**35 (Total)**	**8 (Total)**

Table 5: Per-Report Similarity-vs-Flip Correlation ($C \rightarrow W$). Pearson r demonstrates negligible global correlation between lexical proximity and flip likelihood.

Hop	n_{hub}/n_{tail}	Hub $\lvert\Delta\text{AttLift}\rvert$	Tail $\lvert\Delta\text{AttLift}\rvert$	$C \rightarrow W$ (Hub)	$C \rightarrow W$ (Tail)
d1	17 / 23	0.08519	0.06279	0.0000	0.0870
d2	14 / 7	0.07513	0.03441	0.3571	0.4286
d3	12 / 23	0.03744	0.03670	0.1667	0.2174
d4	13 / 15	0.04365	0.10698	0.2308	0.1333
d5	13 / 16	0.04784	0.09322	0.1538	0.1250

Table 6: **Attention Lift under the Clean-Correct Mask.** Results from the stricter paired-audit subset where the pre-update model predicts the correct token accurately (clean-correct Mask). The same final-layer, first-step attention-lift statistic is used here. The early-hop hub advantage in attention perturbation remains visible at $d1$–$d2$, while the far-hop tail rebound persists at $d4$–$d5$. This table is used to bound the strength of the mechanistic claim rather than to establish a universal law.

raw paired audit: the hub-sourced update has a larger attention perturbation at early hops ($d1$–$d2$), while the tail-sourced update rebounds at distant hops ($d4$–$d5$). At $d3$, the two are nearly identical (0.03744 vs. 0.03670), which further argues against any monotonic "hub always dominates" interpretation.